



VPC Design for EKS

An EKS cluster runs in a VPC you build. Route tables decide which subnets are public, two zones keep it survivable, subnet size caps how much you can run, and tags tell EKS where to put load balancers.

IN THIS LECTURE

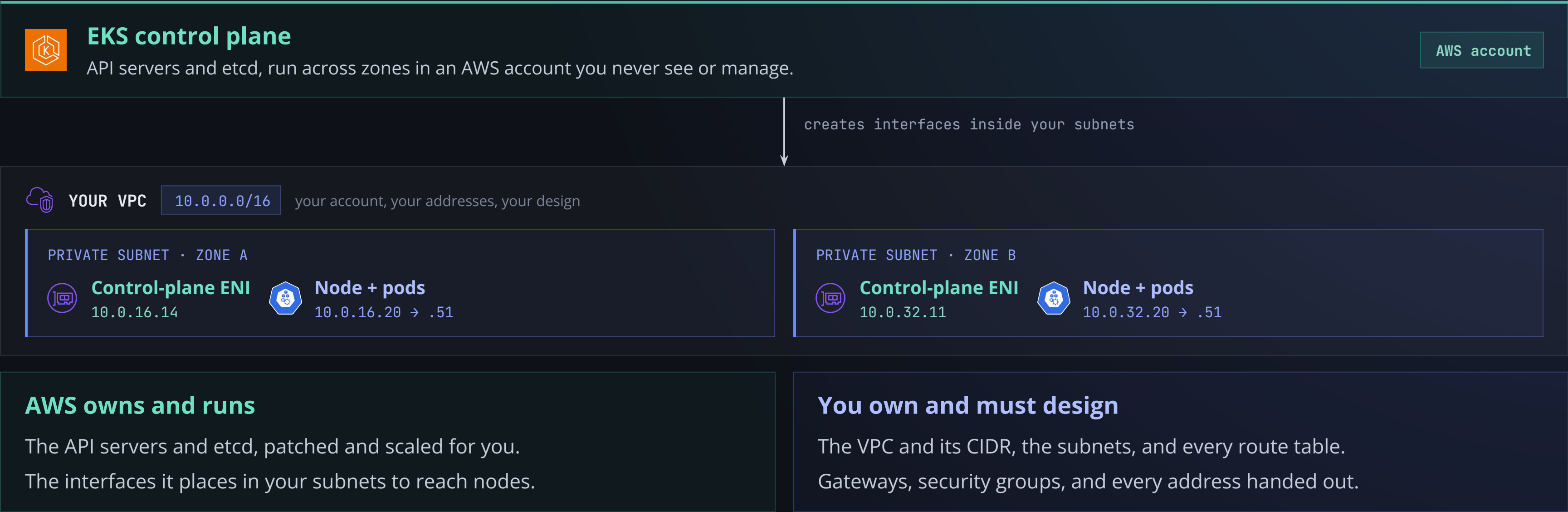
- 01 The cluster's VPC**
Interfaces and pod IPs come from your space.
- 02 Subnets and route tables**
One route decides public or private.
- 03 Zones and address space**
Two zones, and enough IPs to grow into.
- 04 NAT gateway and subnet tags**
Egress out, and tags that place load balancers.
- 05 The assembled network**
Every choice in one picture.



WHERE THE CLUSTER LIVES

The control plane reaches into your VPC

AWS runs the API servers in its own account, then places network interfaces inside your subnets to reach your nodes. Every address involved comes out of your range.



THE ACTUAL MECHANISM

Public or private is one line in a route table

Both tables carry the same local route. Only the default route differs, and changing its target is what flips a subnet from public to private.



Public subnet

reachable, and reaches out directly

172.16.0.0

172.16.1.0

172.16.2.0

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-0a1b2c3d

✓

The default route points at an internet gateway.

✓

Holds only what genuinely must face the internet.



load balancers · NAT gateway



Private subnet

reaches out, never reachable

172.16.0.0

172.16.1.0

172.16.2.0

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	nat-04e5f6a7

✓

The same default route points at a NAT gateway.

✓

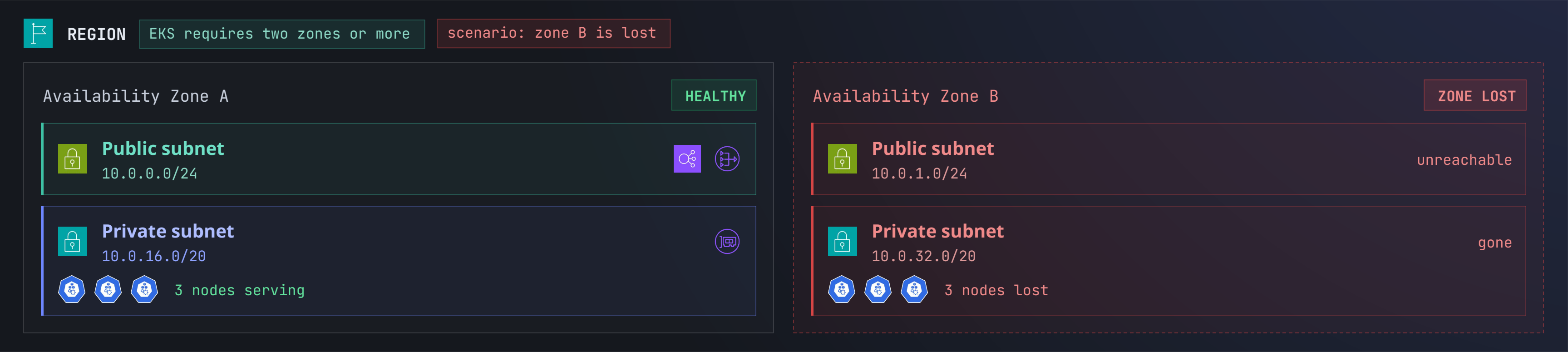
Holds the compute and the data, and most of your addresses.



worker nodes · pods · databases

Run the failure: what a lost zone costs

Zones are physically separate data centers, so a fault in one does not reach the others. Spread across two, and losing a whole zone takes a slice of capacity instead of the cluster.



Still serving

Zone A keeps running, the API stays reachable through its healthy interface, and the load balancer stops sending traffic to dead targets on its own.

Lost

Three nodes and every pod on them, roughly half your capacity, until the surviving zone scales up to absorb the load.

The catch

Had the one shared NAT gateway lived in zone B, every surviving private subnet would now have no way out, and no way to pull an image.

Pod IPs are VPC IPs, so subnet size is a ceiling

Every pod takes a real address from the subnet it runs in. Size the subnet for the pods you will run, not for the nodes you can count today.

Subnet	Usable IPs	What it really holds
/27	27	The practical floor: one load balancer needs a handful.
/24	251	Each public subnet here: a load balancer and a NAT gateway.
/22	1,019	A modest private subnet, until the pods multiply.
/20	4,091	Each private subnet here: room to scale out and upgrade.

The arithmetic

$$\text{usable} = 2^{(32 - \text{prefix})} - 5$$

AWS reserves five addresses in every subnet it creates.

Worked through

1 node + 30 pods = 31 addresses

3 nodes = 93 addresses

A rolling upgrade briefly doubles the node count, and doubles that number with it.



Private subnets get the big ranges

Every node and every pod lives here, so this is where IP exhaustion can happen.

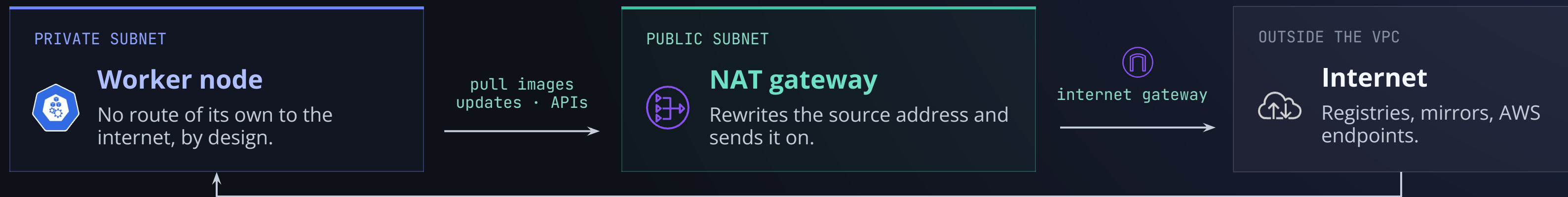


Public subnets stay small

They hold a load balancer and a NAT gateway, so a /24 is already generous.

NAT gateway: a one-way door out

Private nodes still pull images, download updates and call AWS APIs. Inbound isolation, yes; total isolation, no.



Two meters

A charge per hour just to exist, plus a charge per GB processed, and every image pull crosses the second one.

A per-zone resource

One in each zone is resilient. One shared is cheaper, and it takes every zone's egress down with it.

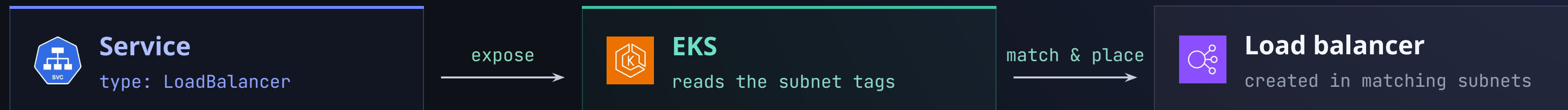
Outbound only

Replies to a connection your node opened come back. Nothing outside can open one inward.

THE CONTRACT

Subnet tags place load balancers

Ask Kubernetes for external access and EKS provisions a load balancer. It cannot guess which subnets to use, so it reads your tags.



Tag public subnets

```
kubernetes.io/role/elb = 1
```

Marks them as the home for internet-facing load balancers.

Tag private subnets

```
kubernetes.io/role/internal-elb = 1
```

Marks them for internal load balancers that never face out.

 **Miss the tags** and you hit the most common EKS support question there is: the load balancer never appears, or it lands in the wrong place. Set them once, up front, and provisioning just works for the life of the cluster.

Three network tradeoffs to set on purpose

Each choice carries an availability, security or cost consequence. There is no universal answer, only the right one for a lab versus production.

Dimension	Benefits	Requirements and Costs
Availability	More zones and a NAT per zone survive the loss of a whole data center.	More spend and a little more complexity. One shared NAT is cheaper and concentrates the risk.
Security	Private nodes behind NAT keep the attack surface small, with nothing inbound reaching the machines.	You still run a public front door: a deliberate load balancer, plus an API endpoint you narrow or keep private.
Cost	Naming the meters up front keeps the monthly number predictable.	The quiet charges: the NAT's two meters, cross-zone traffic, and public addresses.

Lab-scale choice

cheap and simple

Two zones, one shared NAT gateway, a public API endpoint you narrow by address.

Production choice

resilient and closed

Three zones, a NAT gateway in each, and a private API endpoint reached from inside.



ONE PICTURE

The whole network, assembled

Two zones, four subnets, one gateway in and one way out. Every element here is a decision you can now explain.

